



Week 1: Environment Setup & Robotics Foundations

TURTLE Incubator: SO-ARM Software Stack

Attendance



Program Overview

Attendance



Our Mission

- Develop the next generation of leaders through hands-on robotics projects, competitions, and community service

Quick Facts

- Founded in 2013 the oldest and largest active robotics organization at TAMU
- 450+ active members across the org
- 18 student-led R&D projects, including SI's own SO-ARM software stack

Attendance



SI in One Line

- A hands-on program that takes complete beginners from zero to programming and controlling a real robotic arm (SO-ARM) in 9 weeks

What SI Really Teaches You

- Not just following tutorials you'll learn to read hardware docs, debug real systems, and think like a robotics software engineer
- Every skill maps directly onto TURTLE's Advanced Projects, where this software runs on real competition robots

Attendance



Presence & Conduct

- An officer or lab lead must be present whenever the lab is in use
- Disrespectful or unsafe behavior can result in disciplinary action or being asked to leave

Priority & Privileges

- Project tasks always take priority over personal projects for lab tools and space
- Misuse or unsafe practices will result in lab privileges being revoked

When In Doubt

- If you need training or supervision on a tool, ask an officer
- If common sense conflicts with anything here, ask an officer for help

Your Safety Comes First

- You're surrounded by people and resources who can help. If you're ever unsure, ask before you act
- Stop immediately and get clarification if you feel unsafe with a tool

Tools That Require Training

- Soldering iron, 3D printers, drill press, hand and power tools
- Any officer can provide tool-specific training

Required Safety Gear

- Closed-toe shoes, bottoms covering to the lower leg, a sleeved shirt, and safety glasses
- Loose hair and clothing must be secured up and out of the way



Meet the SI Team

Software Incubator Leads & Instructors

Julia Sopala - Development VP



Email: jusopala18@tamu.edu

Discord: juli.a9847



- BMEN 2027
- Meeting Days: TBD
- Experience:
 - OLSN (Prosthetic hand project)
 - R&D Engineering Internship
 - Biomedical Research Lab Experience
 - Hatchling Instructor
 - Former Hatchling
- Fun facts:
 - I want to be fluent in 4 languages
 - I'm 3/5th of the way there
 - I want to learn how to surf

Kaleb - SI Lead



Email: Hkaleb290@tamu.edu

Discord: [thanos0818](#)



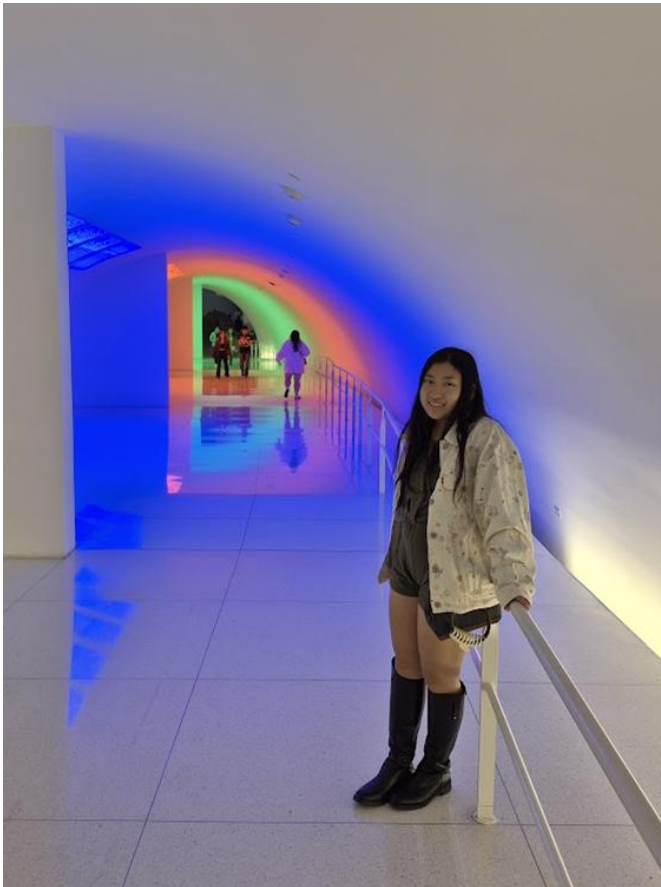
- ENGR (2028)
- Meeting Days: TBD
- Experience:
DIRT Software team lead (now project lead)
just a chill dude
- Fun facts:
rides a skateboard

Kayla Sam - SI Instructor



Email: kaysam@tamu.edu

Discord: kkaayyss



- ENGR 2029
- Meeting Days: TBD
- Experience:
 - FASH Electrical Team Lead
 - FTC Robotics
- Fun facts:
 - I have a twin brother
 - I sing in an A&M Choir
 - I'm living in a temporary apartment 🌸



Mohid Ismail - SI Instructor



Email: mohidismail@tamu.edu

Discord: bean.man.



- ELEN 2027
- Meeting Days:
 - the only one ig
- Experience:
 - VIRT
 - Optics Research Lab Experience
 - Former Hatchling
- Fun facts:
 - Have over 7000 hours on LoL
 - Lost a 3v1 to a goat once

Nathan Mersino - SI Instructor



Email: nathanmersino@tamu.edu
Discord: nathanmersino



- ELEN 2028
- Meeting Days:
 - TBD
- Experience:
 - POBS: Electrical Subteam Lead
 - Former Hatchling
 - Instrumentation Engineering/Transmitter Internship
- Fun facts:
 - In the TAMU Photography Club
 - 1220 Day Duolingo Streak





Icebreakers!

Icebreaker: Name and Fact



How It Works

- Share your name, major, year, and one fun fact about yourself

After Everyone Goes

- Pair up with someone who shared a similar fact and chat for a minute

TURTLE Introductions

- Meet the team leads, learn names, understand how the lab operates

Incubator Goals

- Build the complete software stack of a robotic arm, from zero to autonomous motion

Expectations

- Weekly hands-on deliverables you must demo before leaving each session

Semester Roadmap

- 9 weeks: setup → Python → kinematics → control → estimation → planning → ROS 2

SO-ARM Project Overview

- Every topic this semester maps directly to making the SO-ARM move smarter

Safety & Rules

- Always power down before touching wiring. Never force a joint. Ask before you unplug anything.



Robotics Software Architecture

What Makes a Robot Intelligent?

A robot without software is just a sculpture.

Three things every intelligent robot must do:

- Sense: read the world through cameras, encoders, and IMUs
- Decide: algorithms process that data and choose what action to take
- Act: motors, servos, and grippers execute the decision physically

This sense → decide → act loop runs hundreds of times per second.

The faster and more accurate it runs, the more intelligent the robot appears.

Software vs. Hardware Responsibilities



Hardware does:

- Move joints, resist gravity, handle electrical signals, enforce physical limits

Software does:

- Read sensors, compute what to do, send commands, catch errors, log data

The rule:

- Hardware sets the ceiling. Software determines how close you get to it.
- Bad software on good hardware = broken robot. Good software on bad hardware = still a broken robot.

Your job this semester is the software side, all of it.

Perception

- Reads raw encoder ticks, camera pixels, IMU data and turns them into useful state: "the arm is at 45°"

Planning

- Given a goal ("pick up the block"), figures out the sequence of positions to move through

Control

- Executes the plan by constantly comparing target vs actual position and correcting the error (PID)

Hardware Interface

- The lowest layer translates software commands into actual voltage signals to the motor drivers

We will build each layer from scratch, one week at a time.



Development Environment

Installs: What You Need Today



Linux → We use Docker (no need to wipe your machine)

- Install Docker Desktop: docker.com/products/docker-desktop

Git

- Windows: git-scm.com/download/win | Mac: `brew install git` | Linux: `sudo apt install git`

VS Code

- Download at code.visualstudio.com, then install the Dev Containers extension

Verify everything works:

- `docker --version` `git --version` `code --version`

Linux is an open-source operating system used on 96% of the world's web servers and every major robotics platform.

Why robotics runs on Linux:

- ROS 2 (Robot Operating System) is Linux-native
- Real-time control, USB device access, and serial comms are much easier on Linux
- Most robotics libraries are written and tested for Linux first

Key commands you'll use constantly:

- `ls` `cd` `pwd` `mkdir` `rm` `cat` `grep` `chmod` `sudo` `apt install`

Docker packages an entire Linux environment into a portable container that runs identically on any machine.

Image

- A frozen snapshot of an OS + all installed software. Think of it as a USB stick with a full Linux system.

Container

- A running instance of an image. Isolated from your host, you can't break your laptop from inside it.

Commands you'll use:

- `docker build -f docker/Dockerfile.user -t lerobot-user .` # build the image (first time only)
- `docker run -it --name lerobot_container lerobot-user` # create and enter the container
- `docker exec -it lerobot_container bash` # reopen a shell later

VS Code is our editor. The Dev Containers extension lets you write and run code directly inside Docker.

Install these extensions now:

- Dev Containers: edit files inside Docker as if they were local
- Python: IntelliSense, linting, and integrated debugger
- GitLens: see who changed what and when, directly in the editor
- ROS: syntax highlighting for launch files and .msg/.srv types

Useful shortcuts:

- Ctrl+` open terminal | Ctrl+Shift+P command palette | Ctrl+Click jump to definition

Git saves a complete history of every change to your code. You can always go back.

The three commands you'll use every session:

- `git add .` # stage all changed files
- `git commit -m "msg"` # save a snapshot with a description
- `git push` # upload your commits to GitHub

Branches let you experiment safely:

- `git checkout -b my-feature` # new branch
- `git checkout main` # switch back to main

Pull Requests let teammates review your code before it merges into main.

SO-ARM Playground

What is the SO-ARM?

The SO-ARM100 is a 6-degree-of-freedom desktop robotic arm built from servo motors and 3D-printed parts.

6 degrees of freedom (DoF) means:

- 6 independent joints, each one rotates to position the arm in 3D space
- More DoF = more flexibility in how you reach a target position

What it can do:

- Pick and place objects, follow trajectories, react to sensor feedback in real time

What powers it:

- Feetech STS3215 servo motors, each has a built-in encoder and PID controller on-chip

Clone the repo first: `git clone https://github.com/huggingface/lerobot`

- `src/lerobot/`: core library, robots, teleoperators, datasets, policies
- `robots/so101_follower/`: follower arm hardware wrapper (this is where SO-ARM lives)
- `teleoperators/so101_leader/`: leader arm hardware wrapper
- CLI commands: `lerobot-find-port`, `lerobot-calibrate`, `lerobot-teleoperate`, `lerobot-train`
- Calibration files: saved per arm under `~/.cache/huggingface/lerobot/calibration/`

Everything you touch this semester happens through these CLI commands, not by editing library source code.

Key Commands You Need to Know



lerobot-find-port

- Detects which USB port each arm is connected to. Run this first, one arm at a time.

lerobot-setup-motors

- Assigns a motor ID to each servo, one at a time, before you can calibrate the arm.

lerobot-calibrate

- Records each joint's range of motion and saves it to a calibration file. Run once per arm.

lerobot-teleoperate

- Runs live teleoperation, mapping leader arm movement to the follower in real time.

How the SO-ARM Workflow is Organized



The workflow has three levels: understand which level you're working at:

Level 1: Hardware (Feetech servos on the bus)

- Speaks directly to servo hardware over serial. Reads raw encoder ticks, writes target positions.

Level 2: Calibration Data (JSON file per arm)

- Stores each joint's motor ID, drive direction, and range of motion for one specific arm.

Level 3: CLI Commands (setup-motors, calibrate, teleoperate)

- User-facing. Reads and writes the calibration data to set up, calibrate, and run the arm.

When something breaks, start at Level 3 and trace down. Don't start at the hardware.

This command is the heart of setup. Three phases you should understand:

1. Connect

- Opens the USB serial port at the port you passed and pings each motor to verify it responds.

2. Record range of motion

- Prompts you to move each joint through its full range, one at a time, and records the min and max.
- Prompt shown: `"Move wrist_flex through its entire range of motion, then press Enter"`

3. Save

- Writes each joint's homing offset and range to a calibration JSON file named after your --id.

Understanding the Calibration File

`~/.cache/huggingface/lerobot/calibration/robots/so_follower/<id>.json` stores one entry per joint:

- `"wrist_flex": {`
- `"id": 4,` the motor ID set during `lerobot-setup-motors`
- `"drive_mode": 0,` flip to 1 if this joint moves backward vs. its twin arm
- `"homing_offset": -142,` set automatically during calibration
- `"range_min": -100.0,` lowest angle reached during calibration
- `"range_max": 100.0` highest angle reached during calibration

You won't normally edit this file by hand, except to flip `drive_mode` when a joint is inverted.

Running Existing Examples: Step by Step



Step 1 Connect the arm

- Plug in USB cable, power on the arm, verify with: `lerobot-find-port`

Step 2 Start the container

- `docker build -f docker/Dockerfile.user -t lerobot-user . && docker run -it --name lerobot_container lerobot-user`

Step 3 Run teleoperation

- `lerobot-teleoperate --robot.type=so101_follower --robot.port=COM3 --robot.id=follower_arm --teleop.type=so101_leader --teleop.port=COM7 --teleop.id=leader_arm`

Step 4 What you should see

- Terminal prints joint positions at ~30 Hz. Moving the leader arm moves the follower arm in real time.

Step 5 Stop cleanly

- Ctrl+C in terminal. The arm will hold its last position. Power off before repositioning by hand.

The robot streams state data continuously while connected. Here's how to read it in Python:

- `from lerobot.robots.so101_follower import SO101Follower, SO101FollowerConfig`
- `config = SO101FollowerConfig(port='COM3', id='follower_arm')`
- `robot = SO101Follower(config)`
- `robot.connect()`

- `# Read all joint positions (returns a dict, degrees)`
- `observation = robot.get_observation()`
- `print(observation) # e.g. {'shoulder_pan.pos': 12.3, 'elbow_flex.pos': 90.2, ...}`

- `# Each key in the dict is a joint name plus .pos`
- `print(observation['wrist_flex.pos'])`

Sending a position command tells each joint where to go. The on-chip PID handles getting there.

- `# Move to a specific pose (all angles in degrees)`
- `action = {`
- `'shoulder_pan.pos': 0.0, # centered`
- `'shoulder_lift.pos': -45.0, # tilted back`
- `'elbow_flex.pos': 90.0, # elbow bent 90 degrees`
- `'wrist_flex.pos': 0.0,`
- `'wrist_roll.pos': 0.0,`
- `'gripper.pos': 20.0, # open`
- `}`
- `robot.send_action(action)`

Three safe places to modify behavior, from safest to most impactful:

1 CLI flags (safest)

- Change `--robot.port`, `--robot.id`, or which arm type you pass. No code changes needed.

2 More CLI arguments (safe)

- `lerobot-teleoperate` also accepts `--fps` (control rate) and `--display_data` (live plots)
- `lerobot-teleoperate --robot.type=so101_follower --robot.port=COM3 --robot.id=follower_arm --teleop.type=so101_leader --teleop.port=COM7 --teleop.id=leader_arm --fps=10 --display_data=true`

3 Calibration file directly (careful)

- Edit the JSON calibration file to flip `drive_mode` or nudge a `homing_offset`
- Only do this if calibration alone doesn't fix the problem, and re-check with teleoperate after



Demonstration

Demonstration: Move SO-ARM Using Provided Software



Watch carefully: you will repeat every step of this in the hands-on.

What we'll show:

- 1. Connect arm, verify USB port, launch Docker container
- 2. Run teleoperation script and move the arm by hand on the leader
- 3. Print live joint positions to terminal while arm moves
- 4. Send a hardcoded position command and watch the arm move to it
- 5. Read motor current to see which joints are under load

Demonstration: Exploring Robot States and Commands



After moving the arm, we'll dig into what the robot is actually reporting:

Present_Position

- Each joint's current angle in degrees. This is ground truth for where the arm is.

Present_Velocity

- How fast each joint is moving. Near zero = arm is still. High value = arm is swinging fast.

Present_Current

- Electrical current drawn. High current on a stationary joint = it's fighting gravity or hitting a limit.

Hardware Error Status

- Non-zero means a motor fault. Overload, overheat, or encoder error. Always check this first when something is wrong.



Hands-On

Goal: get the SO-ARM codebase onto your machine and verify it looks right.

1. Open a terminal (Git Bash on Windows, Terminal on Mac/Linux)

2. Clone the repo

- `git clone https://github.com/huggingface/lerobot`
- `cd lerobot`

3. Explore the structure

- `ls lerobot/` # see top-level folders
- `ls src/lerobot/robots/` # find `so101_follower`
- `ls ~/.cache/huggingface/lerobot/calibration/` # see saved calibration files (after Week 1 setup)

4. Verify your USB port (arm must be plugged in and powered)

- `lerobot-find-port` # works the same on Windows, Mac, and Linux

Hands-On: Launch Development Environment



Goal: get the Docker container running and open VS Code inside it.

1. Start the container

- `docker build -f docker/Dockerfile.user -t lerobot-user .`
- `docker run -it --name lerobot_container lerobot-user`

2. Reopen a shell later if you close the terminal

- `docker exec -it lerobot_container bash`
- You should now see a prompt like: `user_lerobot@8d48309069b0:/lerobot$`

3. Verify the install

- `python -c "import lerobot; print(lerobot.__version__)"`

4. Open in VS Code (optional but recommended)

- `Ctrl+Shift+P` → 'Dev Containers: Attach to Running Container' → select `lerobot_container`

Hands-On: Run First SO-ARM Program



Goal: make the arm move and read live joint data from Python.

1. Run teleoperation

- `lerobot-teleoperate --robot.type=so101_follower --robot.port=COM3 --robot.id=follower_arm \`
- `--teleop.type=so101_leader --teleop.port=COM7 --teleop.id=leader_arm`
- Move the leader arm, and the follower should mirror it in real time.

2. Read joint positions in a separate Python script

- `from lerobot.robots.so101_follower import SO101Follower, SO101FollowerConfig`
- `config = SO101FollowerConfig(port='COM3', id='follower_arm')`
- `robot = SO101Follower(config)`
- `robot.connect()`
- `print(robot.get_observation())`

3. Observe the numbers change as you move joints by hand (power off first for manual moves)



Deliverable

Deliverable: Successfully Run and Modify an Existing SO-ARM Example



You must demonstrate all four of the following to a lab lead before leaving:

1 Repository cloned and structure explored

- Show us the calibration JSON for your follower arm and explain in one sentence what `drive_mode` does

2 Docker container running

- `docker ps` shows `lerobot_container` as 'Up', you can exec into it

3 Teleoperation working

- `lerobot-teleoperate` runs, arm responds to leader, joint positions print to terminal

4 Parameter modified and explained

- Change `--fps` in the `lerobot-teleoperate` command, re-run, and explain what changed and why